



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	PRUEBA PRACTICA TOWER DEFENSE: LISTAS SECUENCIALES, DOBLE Y CIRCULAR EN JAVA
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 - B
Alumnos participantes:	Chico Yunda Juan Carlos Guanoquiza Aguaguña Justin Alexander Tacuri Santillan Mónica Sara
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

1.1 Actividades por desarrollar

Objetivo General:

Diseñar e implementar un mini sistema de juego tipo Tower Defense en Java que demuestre el uso correcto de estructuras de datos lineales implementadas manualmente (lista secuencial, lista doblemente enlazada y lista circular), sin recurrir a las colecciones de java.util

Objetivos específicos:

1. Implementar la lista secuencial basada en arreglo (ListaSecuencialTorres) con sus operaciones de inserción, eliminación por id, búsqueda por id, mostrado y conteo de torres activas, respetando el límite fijo de MAX_TORRES.
2. Desarrollar la lista doblemente enlazada (ListaDobleEnemigos) con referencias a primero y último nodo, implementando la inserción al final, la eliminación de enemigos destruidos y el recorrido en ambos sentidos (adelante y atrás) sin dejar nodos huérfanos.
3. Construir la lista circular simplemente enlazada (ListaCircularOleadas) con referencia al último nodo, de modo que el ciclo de oleadas avance y se reinicie correctamente sin recurrir en ningún momento a comparaciones contra null.

Tower Defense: Listas Secuencial, Doble y Circular en Java

1. Ejercicio

Prueba practica individual/grupal de Estructura de Datos en Java, con IDE libre (VS Code, NetBeans, IntelliJ IDEA o Eclipse). Tiempo sugerido: 2 a 3 horas.

Contexto: el estudiante debe desarrollar un programa en Java inspirado en un juego de tipo Tower Defense, donde el jugador defiende un territorio de oleadas de enemigos que avanzan por una ruta hacia una base, usando torres defensivas que atacan de forma automatica a los enemigos que entran en su rango.

Objetivo academico: disenar e implementar un mini sistema de juego en Java que demuestre el uso correcto de estructuras de datos lineales (implementadas a mano, sin colecciones de java.util), manejo de nodos y logica basica de simulacion por turnos.

Estructuras de datos obligatorias: (1) lista secuencial basada en arreglo para las torres defensivas (Torre: id, nombre, tipo, posicion, danio, rango, costo; operaciones insertar, eliminar por id, buscar por id, mostrar todas, contar activas); (2) lista doblemente enlazada con referencia a primero y ultimo para los enemigos activos (NodoEnemigo sobre Enemigo: id, tipo, vida, velocidad, posicion, recompensa; operaciones insertar al final, eliminar destruido, buscar por id, recorrer adelante y atras, actualizar posicion); (3) lista simplemente enlazada circular con referencia a ultimo para las oleadas (NodoOleada sobre Oleada: idOleada, cantidadEnemigos, tipoEnemigo, vidaBase, velocidadBase; operaciones registrar, mostrar, avanzar a la siguiente oleada y reiniciar el ciclo).



Reglas del juego: el territorio es una ruta lineal de posiciones (0 a 20); los enemigos aparecen en la posición 0 y avanzan según su velocidad en cada turno; las torres atacan automáticamente a los enemigos dentro de su rango; un enemigo con vida 0 se elimina de la lista de activos; un enemigo que llega al final del camino resta una vida al jugador; la partida termina cuando el jugador pierde todas sus vidas o se completan todas las oleadas definidas. Menu requerido (10 opciones): registrar torre defensiva, mostrar torres registradas, eliminar torre, registrar oleada, mostrar oleadas, iniciar siguiente oleada, avanzar turno, mostrar enemigos activos, mostrar estado general del juego, salir.

Restricciones técnicas: lenguaje obligatorio Java; estructuras implementadas manualmente con arreglos y referencias entre objetos; prohibido usar ArrayList, LinkedList, Queue, Deque o colecciones similares; se permite Scanner y String para entrada de datos; clases separadas en archivos .java, con TowerDefenseApp como clase con el método main.

Entregables: archivos fuente .java compilables y ejecutables; capturas de pantalla de la ejecución; breve explicación del uso de cada estructura de datos; diagrama simple de clases o de estructuras utilizadas; invitar al docente como colaborador del repositorio de GitHub.

2. Solución propuesta

Se crean tres estructuras de datos independientes, cada una responsable de una sola colección del juego: ListaSecuencialTorres (arreglo estático) para las torres, ListaDobleEnemigos (lista doblemente enlazada con primero y último) para los enemigos activos, y ListaCircularOleadas (lista simple circular con referencia a último) para el ciclo de oleadas. La clase Juego orquesta las tres estructuras e implementa la lógica de turnos (generar, mover, atacar, eliminar, descontar vidas), y TowerDefenseApp expone el menú de 10 opciones sobre esa clase.

Clase	Responsabilidad
Torre	Guarda los datos de una torre defensiva (id, nombre, tipo, posición, daño, rango, costo) y calcula si un enemigo está dentro de su rango
Enemigo	Guarda los datos de un enemigo (id, tipo, vida, velocidad, posición, recompensa), recibe daño y avanza por la ruta
Oleada	Guarda la definición de una oleada (idOleada, cantidadEnemigos, tipoEnemigo, vidaBase, velocidadBase)
NodoEnemigo	Envuelve a un Enemigo con las referencias anterior y siguiente para la lista doble
NodoOleada	Envuelve a una Oleada con la referencia siguiente para la lista circular
ListaSecuencialTorres	Arreglo estático de Torre con un contador tope; implementa insertar, eliminar por id, buscar por id, mostrar y contar
ListaDobleEnemigos	Lista doblemente enlazada de NodoEnemigo con primero y último; implementa insertar al final, eliminar, buscar, recorrer en ambos sentidos y actualizar posiciones
ListaCircularOleadas	Lista simplemente enlazada circular de NodoOleada con referencia a último; implementa registrar, mostrar, avanzar a la siguiente oleada y reiniciar el ciclo
Juego	Mantiene las tres estructuras, las vidas del jugador y el turno actual; ejecuta la lógica de cada turno y del avance de oleadas
TowerDefenseApp	Clase con el método main; muestra el menú de 10 opciones y llama a los métodos públicos de Juego

3. TDA ListaSecuencialTorres

Valores:

- Torre[] torres (arreglo de tamaño fijo, MAX_TORRES = 20)
- int tope (cantidad de torres realmente ocupadas del arreglo)

Operaciones:

- insertarTorre : Torre --> boolean



- eliminarTorrePorId : int --> boolean
- buscarTorrePorId : int --> Torre
- mostrarTodas : Void --> Void
- contarActivas : Void --> int

3. TDA ListaDobleEnemigos

Valores:

- NodoEnemigo primero
- NodoEnemigo ultimo
- int cantidad

Operaciones:

- insertarFinal : Enemigo --> Void
- eliminarDestruido : int --> boolean
- buscarPorId : int --> Enemigo
- recorrerAdelante : Void --> Void
- recorrerAtras : Void --> Void
- actualizarPosiciones : Void --> Void

3. TDA ListaCircularOleadas

Valores:

- NodoOleada ultimo (el siguiente del ultimo apunta siempre al primero)
- NodoOleada actual (proxima oleada por iniciar)
- int cantidad

Operaciones:

- registrarOleada : Oleada --> Void
- mostrarOleadas : Void --> Void
- avanzarSiguienteOleada : Void --> Oleada
- reiniciarCiclo : Void --> Void

4. Relación entre requisitos y código

Tabla que conecta cada requisito del enunciado (arreglo estático, primitivos, herencia, polimorfismo, etc.) con la parte del código donde se aplica. Marcar "No aplica" si el enunciado no lo pide.

Requisito	Aplicación
Lista secuencial (arreglo) para torres	ListaSecuencialTorres: Torre[] torres, int tope; negocio/ListaSecuencialTorres.java
CRUD de torres (insertar, eliminar, buscar, mostrar, contar)	Metodos de ListaSecuencialTorres: insertarTorre, eliminarTorrePorId, buscarTorrePorId, mostrarTodas, contarActivas
Lista doblemente enlazada con primero y ultimo	ListaDobleEnemigos: NodoEnemigo primero, ultimo; negocio/ListaDobleEnemigos.java
Insertar, eliminar, buscar y recorrer en ambos sentidos (enemigos)	Metodos de ListaDobleEnemigos: insertarFinal, eliminarDestruido, buscarPorId, recorrerAdelante, recorrerAtras
Lista simplemente enlazada circular con referencia a ultimo	ListaCircularOleadas: NodoOleada ultimo; negocio/ListaCircularOleadas.java



Registrar, mostrar, avanzar y reiniciar el ciclo de oleadas	Metodos de ListaCircularOleadas: registrarOleada, mostrarOleadas, avanzarSiguienteOleada, reiniciarCiclo
No usar ArrayList, LinkedList, Queue, Deque ni colecciones similares	Las tres estructuras se implementan a mano con arreglos y referencias entre objetos (Torre[], NodoEnemigo, NodoOleada)
Logica de turnos (mover, atacar, eliminar, descontar vidas)	Juego.avanzarTurno() en negocio/Juego.java
Menu con las 10 opciones del enunciado	TowerDefenseApp.main() en app/TowerDefenseApp.java
Herencia / Polimorfismo	No aplica: el enunciado no pide una jerarquia de clases

5. Pasos de construcción

Paso 1: Importar las librerías necesarias

```
import java.util.Scanner;
```

Se importa Scanner en TowerDefenseApp.java para leer las opciones del menu y los datos que ingresa el jugador (nombre y tipo de torre, cantidad de enemigos, etc.). No se usa ninguna otra libreria de colecciones.

Paso 2: Crear las clases del modelo

Torre.java

```
package modelo;

public class Torre {
    private int id;
    private String nombre;
    private String tipo;
    private int posicion;
    private int danio;
    private int rango;
    private int costo;

    public Torre(int id, String nombre, String tipo, int posicion,
                 int danio, int rango, int costo) {
        this.id = id;
        this.nombre = nombre;
        this.tipo = tipo;
        this.posicion = posicion;
        this.danio = danio;
        this.rango = rango;
        this.costo = costo;
    }
}
```



```
public boolean dentroDeRango(int posicionEnemigo) {  
    return Math.abs(posicionEnemigo - this.posicion) <=  
this.rango;  
}  
}
```

Datos de una torre: posicion fija en la ruta, danio y rango para saber si un enemigo esta a tiro, costo. dentroDeRango() es el unico comportamiento propio.

Enemigo.java

```
package modelo;  
  
public class Enemigo {  
    private int id;  
    private String tipo;  
    private int vida;  
    private int velocidad;  
    private int posicion;  
    private int recompensa;  
  
    public Enemigo(int id, String tipo, int vida, int velocidad,  
                    int posicion, int recompensa) {  
        this.id = id;  
        this.tipo = tipo;  
        this.vida = vida;  
        this.velocidad = velocidad;  
        this.posicion = posicion;  
        this.recompensa = recompensa;  
    }  
  
    public void recibirDanio(int danio) {  
        this.vida -= danio;  
        if (this.vida < 0) this.vida = 0;  
    }  
  
    public boolean estaDestruido() { return this.vida <= 0; }  
    public void avanzar() { this.posicion += this.velocidad; }  
}
```

Datos de un enemigo: vida y velocidad determinan cuanto aguanta y cuanto avanza cada turno. recibirDanio()/avanzar() son las dos acciones que Juego le aplica en cada turno.

Oleada.java

```
package modelo;  
  
public class Oleada {  
    private int idOleada;
```



```
private int cantidadEnemigos;
private String tipoEnemigo;
private int vidaBase;
private int velocidadBase;

public Oleada(int idOleada, int cantidadEnemigos, String
tipoEnemigo,
                int vidaBase, int velocidadBase) {
    this.idOleada = idOleada;
    this.cantidadEnemigos = cantidadEnemigos;
    this.tipoEnemigo = tipoEnemigo;
    this.vidaBase = vidaBase;
    this.velocidadBase = velocidadBase;
}
}
```

Solo datos, sin comportamiento: cuantos enemigos, de que tipo, con que vida y velocidad base arrancan en esa oleada.

NodoEnemigo.java

```
package modelo;

public class NodoEnemigo {
    private Enemigo dato;
    private NodoEnemigo anterior;
    private NodoEnemigo siguiente;

    public NodoEnemigo(Enemigo dato) {
        this.dato = dato;
    }
}
```

Envoltorio para la lista doble: guarda el Enemigo y las dos referencias (anterior, siguiente) que arman el enlace en ambos sentidos.

NodoOleada.java

```
package modelo;

public class NodoOleada {
    private Oleada dato;
    private NodoOleada siguiente;

    public NodoOleada(Oleada dato) {
        this.dato = dato;
    }
}
```

Envoltorio para la lista circular: guarda la Oleada y solo siguiente, porque esta lista es simplemente enlazada (no hace falta anterior).



Paso 3: Crear las tres estructuras del negocio

ListaSecuencialTorres.java

```
private Torre[] torres;
private int tope;

public boolean insertarTorre(Torre torre) {
    if (tope >= MAX_TORRES) return false;
    if (existePosicion(torre.getPosicion())) return false;
    torres[tope] = torre;
    tope++;
    return true;
}
```

Lista secuencial: arreglo fijo de Torre con un contador tope. insertarTorre valida capacidad maxima y que no haya otra torre en esa posicion antes de agregar.

ListaDobleEnemigos.java

```
private NodoEnemigo primero;
private NodoEnemigo ultimo;
private int cantidad;

public void insertarFinal(Enemigo enemigo) {
    NodoEnemigo nuevo = new NodoEnemigo(enemigo);
    if (primero == null) {
        primero = nuevo;
        ultimo = nuevo;
    } else {
        nuevo.setAnterior(ultimo);
        ultimo.setSiguiente(nuevo);
        ultimo = nuevo;
    }
    cantidad++;
}
```

Lista doblemente enlazada: mantiene primero y ultimo. insertarFinal conecta el nuevo nodo con anterior/siguiente, lo que permite despues recorrer la lista en ambos sentidos.

ListaCircularOleadas.java

```
private NodoOleada ultimo;
private NodoOleada actual;
private int cantidad;

public void registrarOleada(Oleada oleada) {
    NodoOleada nuevo = new NodoOleada(oleada);
    if (ultimo == null) {
        nuevo.setSiguiente(nuevo);
        ultimo = nuevo;
    }
}
```



```
        actual = nuevo;
    } else {
        nuevo.setSiguiente(ultimo.getSiguiente());
        ultimo.setSiguiente(nuevo);
        ultimo = nuevo;
    }
    cantidad++;
}
```

Lista simplemente enlazada circular: referencia a ultimo, y el siguiente del ultimo siempre apunta al primero. registrarOleada mantiene esa propiedad al insertar, sin comparar nunca contra null.

Paso 4: Implementar las operaciones de cada estructura

```
private void desconectar(NodoEnemigo nodo) {
    NodoEnemigo anterior = nodo.getAnterior();
    NodoEnemigo siguiente = nodo.getSiguiente();
    if (anterior != null) {
        anterior.setSiguiente(siguiente);
    } else {
        primero = siguiente;
    }
    if (siguiente != null) {
        siguiente.setAnterior(anterior);
    } else {
        ultimo = anterior;
    }
    cantidad--;
}
```

Se implementan insertar, eliminar, buscar y recorrer (adelante y atrás) en cada estructura, cuidando los casos borde: eliminar el primero, el ultimo o un nodo intermedio en la lista doble, y no dejar nunca una torre o enemigo huérfano al reconectar referencias.

Paso 5: Implementar la lógica del juego (avanzar turno)

```
public void avanzarTurno() {
    turnoActual++;
    generarEnemigoDeOleada();
    enemigosActivos.actualizarPosiciones();
    aplicarAtaquesDeTorres();
    eliminarEnemigosDestruídos();
    descontarVidasPorEnemigosQueLlegaron();
    mostrarResumenDeTurno();
    verificarFinDePartida();
}
```

La clase Juego usa las tres estructuras para ejecutar, en orden, las acciones que pide el enunciado en cada turno: generar enemigos de la oleada en curso, moverlos, aplicar el dano de las torres que los tengan en rango, eliminar a los destruidos, descontar vidas si alguno llego al final de la ruta, y verificar si la partida termino.



Paso 6: Instanciar y probar en el Main

```
Juego juego = new Juego(3);  
...  
case 7:  
    juego.avanzarTurno();  
    break;
```

TowerDefenseApp crea una única instancia de Juego con 3 vidas iniciales y despacha cada opción del menú a sus métodos públicos, validando siempre que las entradas numéricas sean válidas antes de usarlas (método leerEntero con Scanner.hasNextInt).

6. Tipos de datos utilizados

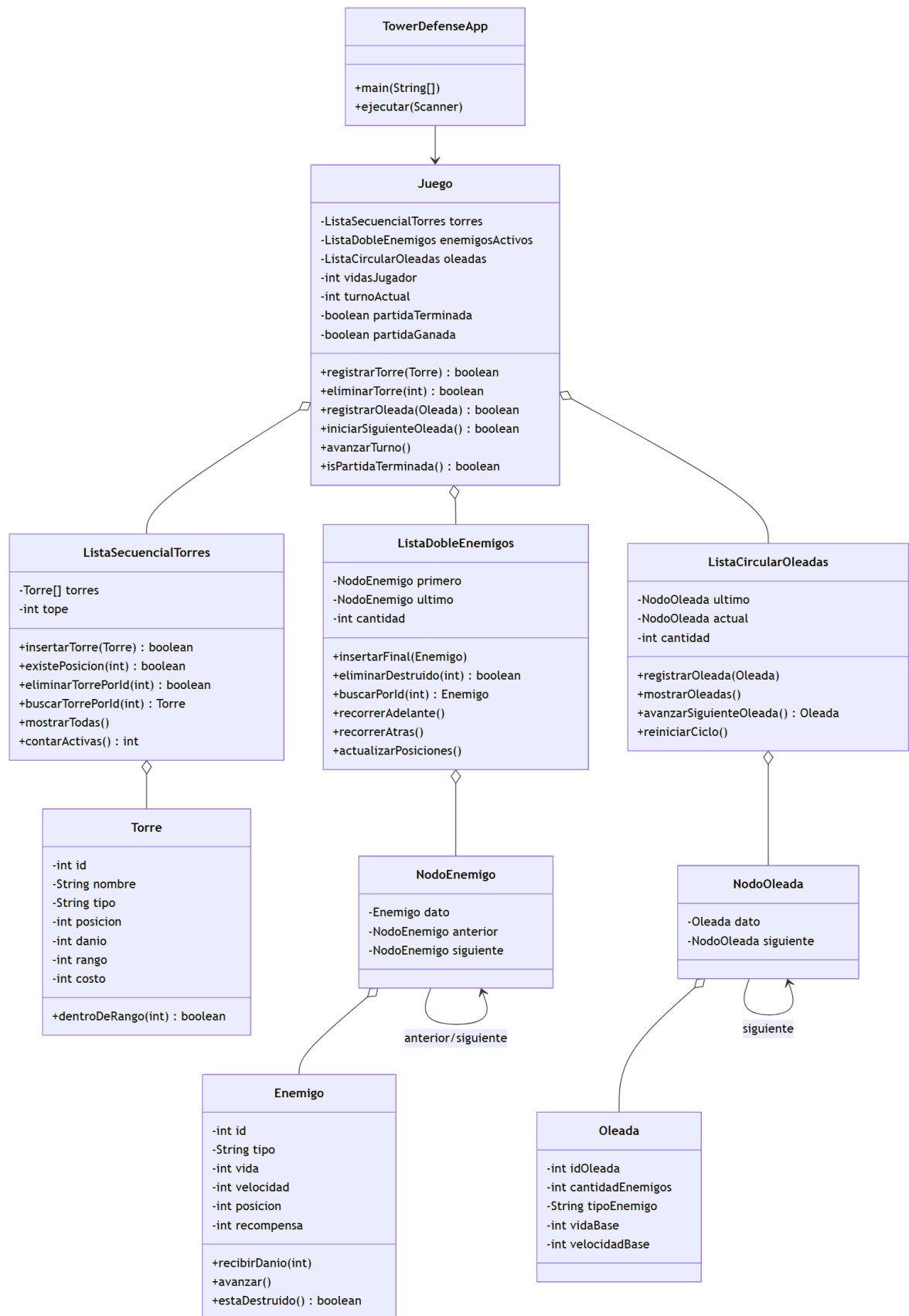
Tipo	Categoría	Variables
int	Primitivo	id, posicion, daño, rango, costo, vida, velocidad, recompensa, tope, cantidad, turnoActual, vidasJugador
String	Referencia (cadena)	nombre, tipo, tipoEnemigo
boolean	Primitivo	resultado de insertarTorre, eliminarTorrePorId, eliminarDestruido, estaDestruido, estaVacía, dentroDeRango
Torre[]	Referencia (arreglo de objetos)	torres, dentro de ListaSecuencialTorres
NodoEnemigo	Referencia (objeto enlazado)	primero, ultimo, anterior, siguiente, dentro de ListaDobleEnemigos y NodoEnemigo
NodoOleada	Referencia (objeto enlazado circular)	ultimo, actual, siguiente, dentro de ListaCircularOleadas y NodoOleada

7. Diagrama de clases

El diagrama se generó a partir del diseño real del código (mismos nombres de clases, atributos y métodos que en Java). Se ve la composición de Juego sobre las tres estructuras (ListaSecuencialTorres, ListaDobleEnemigos, ListaCircularOleadas), y de cada estructura sobre su clase de dominio: ListaSecuencialTorres con Torre (arreglo), ListaDobleEnemigos con NodoEnemigo (que a su vez envuelve a Enemigo), y ListaCircularOleadas con NodoOleada (que envuelve a Oleada). No hay herencia ni polimorfismo porque el enunciado no pide una jerarquía de clases, solo las tres estructuras lineales.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026



8. Especificación del TDA



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026



ListaSecuencialTorres guarda un arreglo fijo de Torre y el número de torres activas (tope). ListaDobleEnemigos guarda los enemigos activos enlazados con referencia a primero y ultimo. ListaCircularOleadas guarda las oleadas enlazadas en un ciclo, con referencia a ultimo y un puntero actual a la próxima oleada por iniciar.

Metodo	Resultado
ListaSecuencialTorres.insertarTorre(t)	Agrega la torre si tope < MAX_TORRES; si no, devuelve false
ListaSecuencialTorres.eliminarTorrePorId(id)	Busca la posición, recorre el arreglo un lugar a la izquierda y decrementa tope
ListaDobleEnemigos.insertarFinal(e)	Crea el nodo, lo enlaza a ultimo y actualiza ultimo (o primero si la lista estaba vacía)
ListaDobleEnemigos.eliminarDestruído(id)	Ubica el nodo y lo desconecta reconectando anterior y siguiente (o primero/ultimo si es un extremo)
ListaCircularOleadas.registrarOleada(o)	Enlaza el nuevo nodo entre ultimo y el primero, y pasa a ser el nuevo ultimo
ListaCircularOleadas.avanzarSiguienteOleada()	Devuelve la oleada en actual y mueve actual a la siguiente del ciclo
Juego.avanzarTurno()	Ejecuta en orden: generar, mover, atacar, eliminar, descontar vidas y verificar fin de partida
Juego.iniciarSiguienteOleada()	Toma la próxima oleada del ciclo circular y la marca como oleada en curso

Invariantes:

- 1. tope nunca supera MAX_TORRES en ListaSecuencialTorres.
- 2. En ListaDobleEnemigos, si primero es null entonces ultimo también es null, y viceversa.
- 3. En ListaCircularOleadas, el siguiente del nodo ultimo siempre apunta al primer nodo registrado (nunca hay null).
- 4. Ningún enemigo con vida menor o igual a 0 permanece en ListaDobleEnemigos después de terminar un turno.

9. Seudocódigo

```
INICIO avanzarTurno
    turnoActual <- turnoActual + 1
    SI hay oleada en curso Y quedan enemigos por generar ENTONCES
        crear enemigo con los datos de la oleada actual
        insertar enemigo al final de la lista de enemigos
    FIN SI
    PARA CADA enemigo en la lista de enemigos
        enemigo.posicion <- enemigo.posicion + enemigo.velocidad
    FIN PARA
    PARA CADA enemigo en la lista de enemigos
        PARA CADA torre en el arreglo de torres
            SI la torre tiene al enemigo en rango ENTONCES
                enemigo.vida <- enemigo.vida - torre.danio
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026



```
FIN SI
FIN PARA
FIN PARA
PARA CADA enemigo en la lista de enemigos
    SI enemigo.vida <= 0 ENTONCES
        eliminar enemigo de la lista
    FIN SI
FIN PARA
PARA CADA enemigo en la lista de enemigos
    SI enemigo.posicion >= longitud de la ruta ENTONCES
        vidasJugador <- vidasJugador - 1
        eliminar enemigo de la lista
    FIN SI
FIN PARA
SI vidasJugador <= 0 0 se completaron todas las oleadas ENTONCES
    partidaTerminada <- verdadero
FIN SI
FIN avanzarTurno
```

10. Prueba de escritorio

Paso	Operación	Estado clave	Resultado
1	registrarTorre(Arquero, pos 3, danio 20, rango 2)	tope = 1	Torre #1 agregada
2	registrarTorre(Canon, pos 8, danio 35, rango 3)	tope = 2	Torre #2 agregada
3	registrarOleada(basico, 3 enemigos, vida 50, vel 1)	oleadas.cantidad = 1	Oleada #1 registrada
4	iniciarSiguienteOleada()	oleadaEnCurso = true, enemigosPorGenerar = 3	Inicia la oleada #1 (basico)
5	avanzarTurno() -- turno 1	enemigo #1 generado en pos 0, avanza a pos 1 y recibe 20 de danio de la torre Arquero	vida enemigo #1 = 30, activos = 1
6	avanzarTurno() -- turno 3	enemigo #1 llega a pos 3 y recibe otro golpe de 20	vida enemigo #1 = 0: Enemigo #1 destruido, activos = 2
7	avanzarTurno() -- turno 5	enemigo #3 llega a pos 3 y recibe el último golpe pendiente	vida enemigo #3 = 0: Enemigo #3 destruido, activos = 0
8	mostrarEstadoGeneral()	turnoActual = 5	vidas = 3, torres = 2, oleadas registradas = 1, oleadas iniciadas = 1



1.2 Resultados obtenidos

Se ejecuto el programa con el caso de prueba sugerido en la guía: dos torres (Arquero en la posición 3 con daño 20 y rango 2; Canon en la posicion 8 con danio 35 y rango 3) y una oleada de 3 enemigos basicos (vida 50, velocidad 1). La Figura 1 muestra la sesion completa en consola: el registro de la torre y la oleada, el inicio de la oleada, el avance de turnos con la eliminación de los enemigos que entran en rango, y el estado final de la partida una vez completadas todas las oleadas definidas.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026



```
TowerDefenseApp - java app.TowerDefenseApp

===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 1
Nombre de la torre: Arquero
Tipo (arquero, canon, mago...): arquero
Posicion en la ruta: 3
Danio: 20
Rango: 2
Costo: 100
Torre registrada con id 1

Elige una opcion: 1
Nombre de la torre: Canon
Tipo (arquero, canon, mago...): canon
Posicion en la ruta: 8
Danio: 35
Rango: 3
Costo: 150
Torre registrada con id 2

Elige una opcion: 4
Cantidad de enemigos en la oleada: 3
Tipo de enemigo: basico
Vida base: 50
Velocidad base: 1
Oleada registrada con id 1

Elige una opcion: 6
Inicia la oleada #1 (basico)

Elige una opcion: 7
--- Turno 1 ---
Vidas del jugador: 3
Enemigos activos: 1

Elige una opcion: 7
--- Turno 2 ---
Vidas del jugador: 3
Enemigos activos: 2

Elige una opcion: 7
--- Turno 3 ---
Enemigo #1 destruido.
Vidas del jugador: 3
Enemigos activos: 2

Elige una opcion: 7
--- Turno 4 ---
Enemigo #2 destruido.
Vidas del jugador: 3
Enemigos activos: 1

Elige una opcion: 7
--- Turno 5 ---
Enemigo #3 destruido.
Vidas del jugador: 3
Enemigos activos: 0
Fin de la partida: se completaron todas las oleadas definidas.

Elige una opcion: 9
Turno actual: 5
Vidas del jugador: 3
Torres registradas: 2
Enemigos activos: 0
Oleadas registradas: 1
Oleadas iniciadas: 1
Partida terminada: true (ganada)
```

Figura 1. Ejecución en consola del caso de prueba (registro, oleada y turnos).



1.3 Conclusiones

1. Trabajar con la lista secuencial de torres permitió entender a fondo el manejo de arreglos estáticos y el control de un contador de posiciones ocupadas (tope), reforzando conceptos de estructuras de datos que con colecciones predefinidas como ArrayList suelen quedar ocultos al programador.
2. La implementación de la lista doblemente enlazada de enemigos fue el mayor reto del ejercicio, ya que exigió cuidar los casos borde al eliminar el primer nodo, el último o uno intermedio; esta experiencia ayudó a comprender por qué este tipo de estructura requiere tanta atención al reconectar referencias entre nodos.
3. Construir la lista circular de oleadas sin usar nunca comparaciones contra null resultó clave para interiorizar la lógica de los ciclos enlazados; observar cómo el puntero "actual" avanza indefinidamente sobre las oleadas registradas consolidó la comprensión de este tipo de estructura de datos.

1.4 Recomendaciones

1. Antes de subir el código a GitHub, agregar pruebas con oleadas más grandes y con enemigos de distinta velocidad, para verificar que la lista doble mantiene el orden de inserción incluso cuando varios enemigos llegan al final de la ruta en el mismo turno.
2. Documentar con comentarios claros los métodos de reconexión de nodos (como desconectar y avanzarSiguienteOleada), para facilitar el mantenimiento futuro del código, dado que son los puntos más sensibles a errores de referencias nulas.
3. Invitar al docente como colaborador del repositorio de GitHub apenas esté creado, y establecer una convención de commits (uno por cada paso de construcción) para dejar trazabilidad del proceso de desarrollo.

1.5 Anexos

Actualizaciones o mejoras del Software

Github : <https://github.com/juanchicoyunda1234/Evaluacion-Practica.git>

Historial de commits del repositorio

Commits		
main		
All users		
All time		
Commits on Sep 14, 2026		
Meti capturas reales en el diseno de la GUI, junto a cada seccion	271b8e2	<>
juanchicoyunda1234 committed 11 minutes ago		
Meti capturas reales de la interfaz grafica en el README	eb655aa	<>
juanchicoyunda1234 committed 11 minutes ago		
Merge branch 'ejecucion'	fde62b4	<>
juanchicoyunda1234 committed 17 minutes ago		
Merge branch 'diagramas-actualizados'	85934d5	<>
juanchicoyunda1234 committed 17 minutes ago		
Agregue la explicacion de las capturas de ejecucion en la interfaz grafica	5eea4f9	<>
juanchicoyunda1234 committed 18 minutes ago		
Agregue la captura del tablero va reiniciado		

Ejecución del programa



```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 1
Nombre de la torre: Torre1
Tipo (arquero, canon, mago...): arquero
Posicion en la ruta: 3
Danio: 20
Rango: 2
Costo: 100
Torre registrada con id 1
```

Ilustración 1 Opción 1

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 2
Torre #1 [Torre1 - arquero] pos=3 danio=20 rango=2 costo=100
```

Ilustración 2 Opción 2

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 3
Id de la torre a eliminar: 1
Torre eliminada.
```

Ilustración 3 Opción 3



```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 4
Cantidad de enemigos en la oleada: 3
Tipo de enemigo: basico
Vida base: 50
Velocidad base: 1
Oleada registrada con id 1
```

Ilustración 4 Opción 4

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 5
Oleada #1 [flecha] cantidad=2 vidaBase=2 velBase=2
```

Ilustración 5 Opción 5

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 6
Inicia la oleada #1 (flecha)
```

Ilustración 6 Opción 6



```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 7
--- Turno 1 ---
Enemigo #1 destruido.
Vidas del jugador: 3
Enemigos activos: 0
```

Ilustración 7 Opción 7

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 8
No hay enemigos activos.
```

Ilustración 8 Opción 8

```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 9
Turno actual: 1
Vidas del jugador: 3
Torres registradas: 1
Enemigos activos: 0
Oleadas registradas: 2
Oleadas iniciadas: 1
Partida terminada: false
```

Ilustración 9 Opción 9



```
===== TOWER DEFENSE =====
1. Registrar torre defensiva
2. Mostrar torres registradas
3. Eliminar torre
4. Registrar oleada
5. Mostrar oleadas
6. Iniciar siguiente oleada
7. Avanzar turno
8. Mostrar enemigos activos
9. Mostrar estado general del juego
10. Salir
Elige una opcion: 10
Cerrando Tower Defense.
```

Ilustración 10 Opción 10

Parte grafica

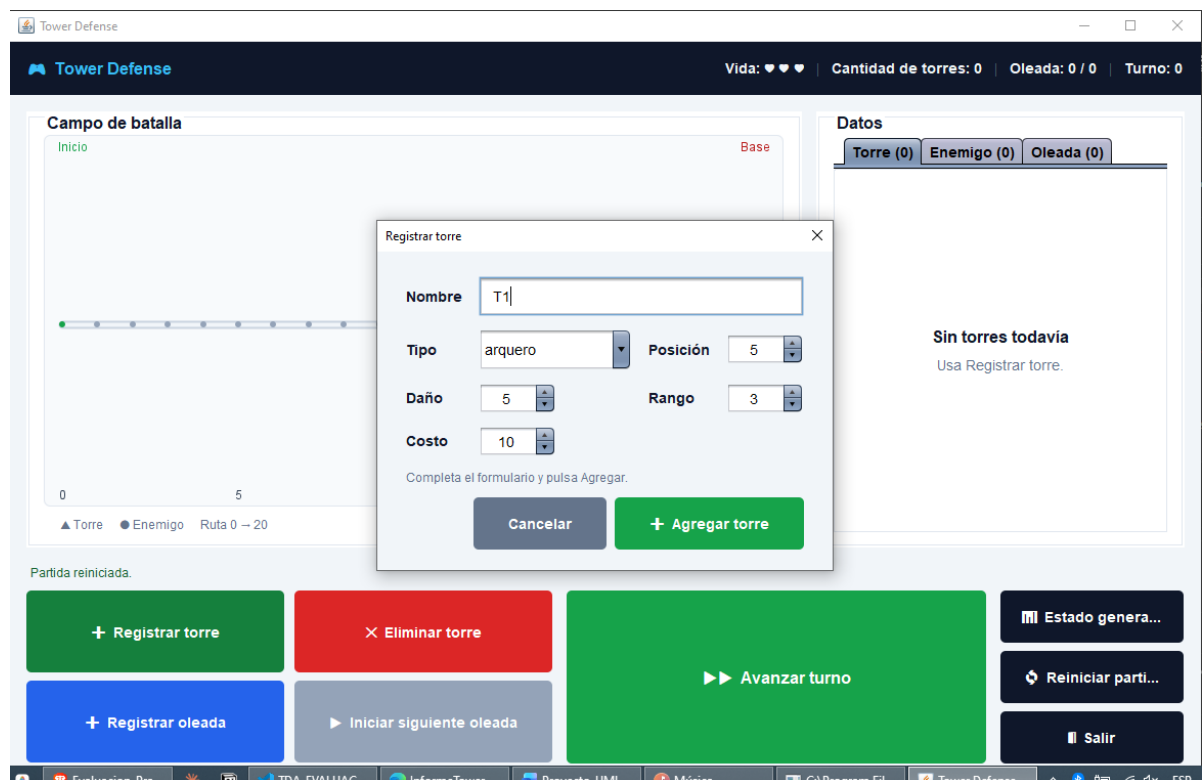


Ilustración 11 Registro de Torre



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026



Tower Defense

Vida: ♥♥♥ | Cantidad de torres: 1 | Oleada: 0 / 0 | Turno: 0

Campo de batalla

Inicio Base

Torre T1

0 5

▲ Torre ● Enemigo Ruta 0 → 20

Registrar oleada

Cantidad: 1 Tipo: goblin

Vida: 5 Velocidad: 1

Completa el formulario y pulsa Agregar.

Cancelar + Agregar oleada

Datos

Torre (1)		Enemigo (0)		Oleada (0)		
Id	No...	Tipo	Pos...	Daño	Ran...	Co...
1	T1	ar...	5	5	3	10

Torre registrada.

+ Registrar torre × Eliminar torre

+ Registrar oleada ▶ Iniciar siguiente oleada

▶▶ Avanzar turno

Estado genera... Reiniciar parti... Salir

Ilustración 12 Registro de Oleadas

Tower Defense

Vida: ♥♥♥ | Cantidad de torres: 1 | Oleada: 0 / 0 | Turno: 0

Campo de batalla

Inicio Base

Torre T1

0 5 10 15 20

▲ Torre ● Enemigo Ruta 0 → 20

Datos

Torre (1)		Enemigo (0)		Oleada (0)		
Id	No...	Tipo	Pos...	Daño	Ran...	Co...
1	T1	ar...	5	5	3	10

Torre registrada.

+ Registrar torre × Eliminar torre

+ Registrar oleada ▶ Iniciar siguiente oleada

▶▶ Avanzar turno

Estado genera... Reiniciar parti... Salir

Ilustración 13 Registro de Batalla



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO 2025 – ENERO 2026

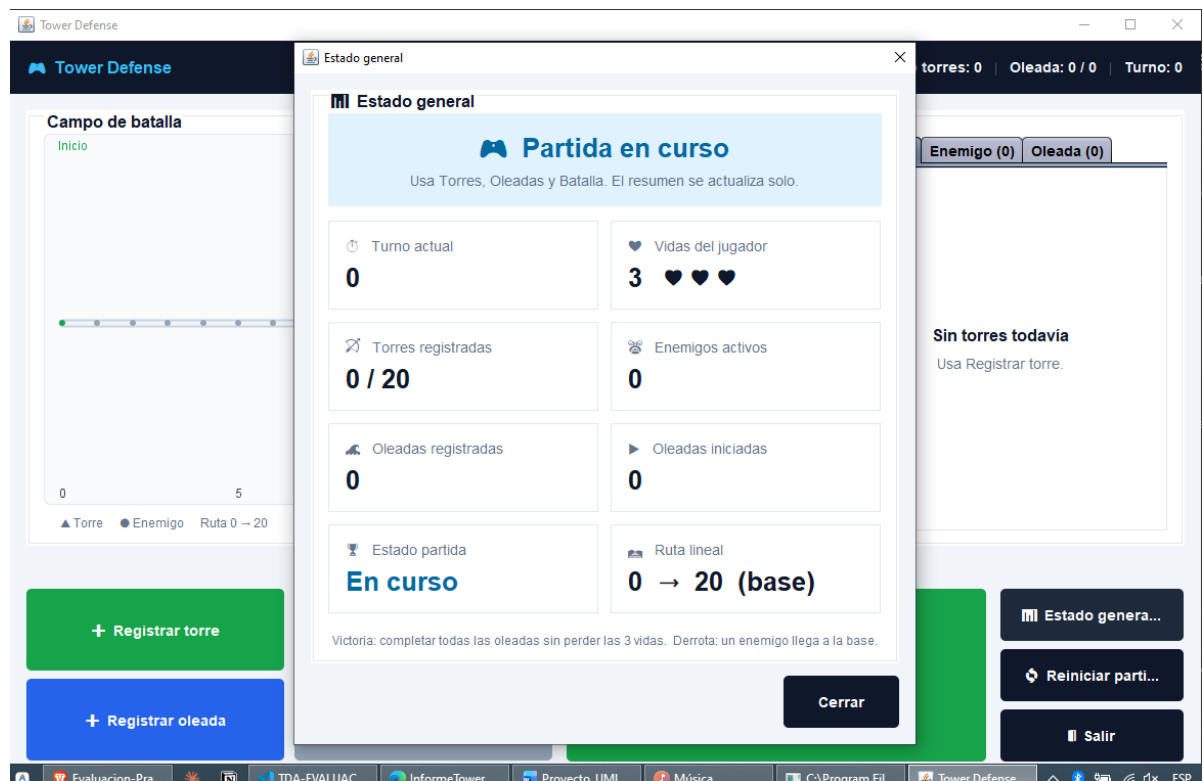


Ilustración 14 Estado de la batalla